

# NEURAL SLAM

Using RGB-D Data and Pose Regression

Pinnamraju Abhishek

March 2025

## Contents

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                     | <b>4</b>  |
| <b>2</b> | <b>Problem Statement</b>                                | <b>5</b>  |
| <b>3</b> | <b>Motivation</b>                                       | <b>5</b>  |
| <b>4</b> | <b>Literature Review</b>                                | <b>6</b>  |
| 4.1      | Neural SLAM Architectures and Enhancements . . . . .    | 6         |
| 4.2      | Integration of Deep Learning in Visual SLAM . . . . .   | 6         |
| 4.3      | Task-Oriented and Domain-Specific Neural SLAM . . . . . | 7         |
| 4.4      | Research Gaps and Limitations . . . . .                 | 7         |
| <b>5</b> | <b>Dataset</b>                                          | <b>8</b>  |
| 5.1      | Tools and Libraries Used . . . . .                      | 8         |
| 5.2      | Conversion Pipeline . . . . .                           | 9         |
| 5.3      | Final Dataset Structure . . . . .                       | 9         |
| <b>6</b> | <b>Methodology</b>                                      | <b>10</b> |
| 6.1      | Dataset Preparation . . . . .                           | 10        |
| 6.2      | Model Architecture . . . . .                            | 10        |
| 6.3      | Training Process . . . . .                              | 11        |
| 6.4      | Post-Processing and Evaluation . . . . .                | 11        |
| 6.5      | Active Neural SLAM (Frame Selection Strategy) . . . . . | 12        |
| <b>7</b> | <b>Experimental Setup</b>                               | <b>12</b> |
| 7.1      | Development Environment . . . . .                       | 13        |
| 7.2      | Libraries and Tools . . . . .                           | 13        |
| 7.3      | Hardware Configuration . . . . .                        | 13        |
| 7.4      | Model Training Configuration . . . . .                  | 14        |
| 7.5      | Active Frame Sampling Strategy . . . . .                | 14        |
| 7.6      | Evaluation Techniques . . . . .                         | 15        |
| <b>8</b> | <b>Evaluation and Results</b>                           | <b>15</b> |



### **Abstract**

Simultaneous Localization and Mapping (SLAM) is a fundamental challenge in robotics and autonomous navigation, where a robot constructs a map of an unknown environment while simultaneously estimating its position within it. Neural SLAM leverages deep learning to enhance traditional SLAM techniques, offering improved robustness and adaptability in complex or dynamic environments. This project explores a deep learning-based SLAM approach using RGB and depth images, trained to regress the camera's pose directly through a convolutional neural network. The model utilizes a ResNet18-based architecture and outputs 6-DoF poses with quaternion representation for orientation. Post-processing includes trajectory alignment and smoothing using Gaussian filters. The system is evaluated using the Absolute Trajectory Error (ATE), achieving a promising result of 0.0156 meters, demonstrating high accuracy in pose estimation. The project also extends into Active Neural SLAM, simulating intelligent exploration through selective frame sampling to optimize information gain and mapping quality.

## 1 Introduction

Simultaneous Localization and Mapping (SLAM) plays a vital role in enabling autonomous agents to navigate unknown environments without relying on external positioning systems like GPS. Traditional SLAM methods rely heavily on handcrafted feature extraction, sensor fusion, and geometry-based estimation, which often struggle in visually complex, low-light, or dynamic environments. Recent advances in deep learning have introduced the concept of Neural SLAM, which integrates learning-based components to estimate camera poses and generate maps from sensory input, such as RGB and depth images. This paradigm shift enables end-to-end training, potentially reducing the dependency on manual feature engineering and enhancing generalizability. In this project, we implement a Neural SLAM pipeline that takes RGB and depth images along with odometry data to predict 6-DoF poses. The backbone model is a ResNet18-based convolutional network trained to minimize pose regression loss, incorporating quaternion normalization for stable orientation learning. The predicted trajectories are aligned using Procrustes analysis and smoothed with Gaussian filters to reduce noise. Further, we explore a simplified version of Active Neural SLAM, a technique that introduces decision-making in frame selection during mapping. Instead of passively processing every frame, Active SLAM aims to select the most informative frames (e.g., every 10th frame or those with significant visual changes), simulating exploration strategies that could be used in real-world robotic systems. This not only improves computational efficiency but can also enhance the mapping quality by focusing on frames that contribute most to reducing uncertainty.

## 2 Problem Statement

This project aims to develop and evaluate a deep learning-based Neural SLAM system capable of predicting 6-DoF poses from RGB-D data in indoor environments. The objectives are:

- To train a ResNet18-based pose regression model using supervised learning from multi-scene data.
- To enhance the predicted trajectory with alignment and smoothing techniques for robust localization.
- To evaluate the accuracy of the system using Absolute Trajectory Error (ATE).
- To simulate Active Neural SLAM by introducing a selective frame sampling strategy that mimics intelligent exploration and reduces computational overhead.

Ultimately, the project seeks to demonstrate how learning-based SLAM can achieve high localization accuracy while also paving the way toward efficient, autonomous exploration.

## 3 Motivation

Robots and autonomous systems increasingly operate in complex, GPS-denied environments where they must build maps and localize themselves with minimal prior knowledge. Traditional SLAM techniques, although widely used, often struggle in dynamic or visually ambiguous scenarios due to their reliance on handcrafted features and geometric constraints. With the rise of deep learning, Neural SLAM has emerged as a powerful alternative—leveraging data-driven models to learn robust pose estimation from raw visual inputs like RGB and depth images. This approach not only improves generalization but also reduces dependence on extensive tuning and hand-engineered features. Moreover, most existing SLAM systems passively process every frame, which can be computationally inefficient. This raises the need for Active Neural SLAM—a system that not only maps and localizes but also decides what to observe next, enabling smarter exploration and better use of resources.

## 4 Literature Review

### 4.1 Neural SLAM Architectures and Enhancements

Neural SLAM has emerged as a cutting-edge approach in the field of Simultaneous Localization and Mapping by integrating neural networks to enhance the capabilities of traditional SLAM systems. Recent studies have introduced algorithm and hardware co-design strategies that aim to reduce computational complexity. These approaches utilize model compression and custom neural accelerators, achieving up to a 31.4% reduction in parameter count and a  $13.8\times$  increase in energy efficiency compared to standard GPU implementations [1].

In parallel, event-based neural SLAM systems have gained traction. Leveraging the high temporal resolution and low-latency data from event cameras, these models can perform accurate pose estimation, depth prediction, and loop closure in real time [2]. Their asynchronous data processing makes them particularly suited for dynamic or low-light environments.

### 4.2 Integration of Deep Learning in Visual SLAM

The integration of deep learning techniques into visual SLAM pipelines has led to substantial advancements in feature extraction, semantic segmentation, and pose regression [3]. End-to-end deep learning models have demonstrated the ability to outperform handcrafted feature-based methods, especially in complex or texture-less environments.

More recently, Neural Radiance Fields (NeRFs) have been adapted for SLAM applications. NeRF-based SLAM allows for implicit scene representation and novel view synthesis, proving to be particularly effective in capturing detailed and photorealistic reconstructions in dynamic scenes [4]. This contributes to both enhanced localization accuracy and richer

environmental understanding.

### 4.3 Task-Oriented and Domain-Specific Neural SLAM

Neural SLAM's versatility is reflected in its adaptability to specific tasks and environments. In Active Neural SLAM (ANS), researchers have developed modular architectures combining global planning, local exploration policies, and neural SLAM backends [5]. These systems enable agents to actively seek out informative viewpoints, optimizing exploration and map coverage.

In underwater navigation, deep learning-based SLAM methods have been tailored to address challenges like poor visibility, lighting variability, and non-rigid scene components. Such systems often incorporate robust sensor fusion and domain-specific pretraining to enhance generalization across diverse underwater environments [6].

### 4.4 Research Gaps and Limitations

Despite the promising advancements, several challenges remain unaddressed in current Neural SLAM systems:

- **High Computational Demands:** Neural SLAM models often require significant processing power and memory, making real-time deployment on low-power or edge devices difficult.
- **Extensive Data Requirements:** Training these models typically necessitates large and diverse datasets, which are often expensive or impractical to collect across all deployment scenarios.



- **Handling of Dynamic Elements:** While some methods filter or adapt to moving objects, achieving consistent and robust mapping in dynamic environments continues to be a major obstacle.
- **System Integration Complexity:** Fusing traditional SLAM components with deep learning models demands careful architectural design to ensure interoperability and real-time performance.
- **Semantic Generalization:** While semantic mapping has improved, models still struggle to generalize object recognition across different domains or unseen environments.

## 5 Dataset

The original dataset was provided as a compressed archive in the form of `office1-1.7-rosbag.tar`, containing ROS2 bag files from indoor environments. To prepare this dataset for training a Neural SLAM model, the following data extraction and conversion pipeline was executed using Python.

### 5.1 Tools and Libraries Used

- **rosbags:** For reading and parsing ROS2 `.bag` files.
- **OpenCV (cv2):** For image manipulation and saving RGB/depth frames.
- **NumPy and Pandas:** For processing pose and odometry data.
- **tarfile:** For decompressing the `.tar` archive.

## 5.2 Conversion Pipeline

1. The `.tar` file was extracted using Python's `tarfile` module.
2. ROS2 bag files (e.g., `office1-1`) were processed using `rosbags.Reader`, and data messages were deserialized using `rosbags.serde`.
3. RGB images were extracted from the topic `/camera/rgb/image_raw`, decoded with NumPy, and saved as PNG files using OpenCV.
4. Odometry data was extracted using the `bagpy` library and stored as CSV files containing 6-DoF ground truth poses: `x`, `y`, `z`, `qx`, `qy`, `qz`, `qw`.
5. Data was organized scene-wise into structured folders (e.g., `office1`, `office2`, etc.) to support training pipelines.

## 5.3 Final Dataset Structure

Each indoor scene folder contains:

- `/rgb/frame_XXXXX.png` — Extracted RGB image frames
- `/depth/depth_XXXXX.png` — Depth frames (if available)
- `odom_data.csv` — Odometry data with time-aligned pose vectors

This structured and synchronized dataset enabled effective training and evaluation of both Neural SLAM and Active Neural SLAM modules.

## 6 Methodology

### 6.1 Dataset Preparation

The dataset consists of multiple indoor scenes (e.g., `office1`, `office2`, etc.) organized into subfolders, each containing:

- RGB images (`/rgb/frame_XXXXX.png`)
- Depth images (`/depth/depth_XXXXX.png`)
- Odometry data in CSV format (`odom_data.csv`), which includes ground truth pose vectors (`x`, `y`, `z`, `qx`, `qy`, `qz`, `qw`)

A custom PyTorch Dataset class named `MultiSceneNeuralSLAMDataset` is used to handle loading and preprocessing of the data. It:

- Iterates through each scene folder
- Loads RGB and depth images for each frame
- Normalizes and resizes the images to  $224 \times 224$  pixels
- Converts depth images into 3-channel format
- Associates each RGB-D pair with its corresponding 7D pose vector (3D position + 4D quaternion orientation)

### 6.2 Model Architecture

The pose regression model is based on the `ResNet18` convolutional backbone. It takes concatenated RGB and depth images as input (6 channels total) and outputs a 7-dimensional pose vector representing:

- **Translation:**  $x, y, z$
- **Orientation (quaternion):**  $q_x, q_y, q_z, q_w$

The model is trained using a combination of L2 loss for both the translation and orientation components.

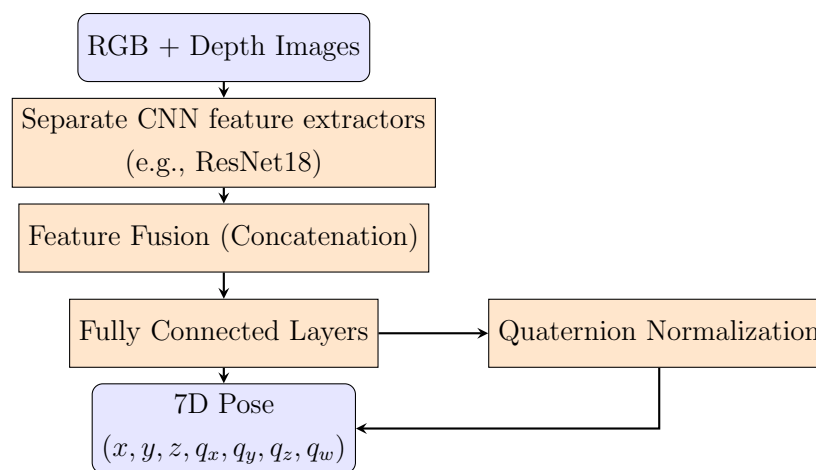


Figure 1: Neural SLAM Model Flowchart

### 6.3 Training Process

- The training loop feeds batched RGB-D images through the model.
- Pose predictions are compared with ground truth using a loss function.
- The model is optimized using a gradient descent-based optimizer (e.g., Adam).
- Quaternion outputs are normalized to ensure valid rotation representations.

### 6.4 Post-Processing and Evaluation

After inference:

- Predicted trajectories are aligned with ground truth using Procrustes analysis to correct for scale and rotation discrepancies.
- A Gaussian filter is applied to smooth the predicted trajectories.
- The system is quantitatively evaluated using Absolute Trajectory Error (ATE), a standard SLAM accuracy metric.

The model achieved an ATE of **0.0156 meters**, indicating high precision in trajectory estimation.

## 6.5 Active Neural SLAM (Frame Selection Strategy)

To simulate Active Neural SLAM, which emphasizes intelligent exploration, a frame selection strategy was implemented:

- Instead of processing every frame, only every 10<sup>th</sup> frame is used.
- This simulates selective observation, reducing computational load while maintaining informative mapping.

This method serves as a simplified but practical version of Active Neural SLAM, where decision-making is introduced to improve mapping efficiency without compromising accuracy.

## 7 Experimental Setup

To conduct the Active Neural SLAM experiments, we utilized a combination of cloud and local resources for model training, trajectory inference, and evaluation.

## 7.1 Development Environment

- **Primary Platform:** Google Colab (for training and visualization using GPUs when available)
- **Local Platform:** VS Code running on Windows 10/11 for offline evaluation and backup execution when GPU time expired
- **Programming Language:** Python 3.9+

## 7.2 Libraries and Tools

- **Deep Learning Framework:** PyTorch (ResNet18-based pose regressor)
- **Image Processing:** OpenCV (for reading and preprocessing RGB and depth frames)
- **Data Processing:** NumPy, Pandas (for loading and manipulating IMU and odometry data)
- **Trajectory Alignment:** SciPy (Procrustes analysis), custom Kabsch implementation
- **Visualization:** Matplotlib (for static and animated 3D plots), Seaborn (optional for stylistic consistency)
- **Smoothing:** SciPy Gaussian filters for post-processing trajectories

## 7.3 Hardware Configuration

- **Cloud (Google Colab):** Tesla T4/P100 GPU, 13GB RAM
- **Local Machine:** Intel i5 CPU, 8GB RAM, integrated GPU (CPU fallback when Colab GPU quota expired)

## 7.4 Model Training Configuration

- **Input Data:** RGB and Depth images resized to  $224 \times 224$
- **Output Format:** 6-DOF pose —  $(x, y, z, q_x, q_y, q_z, q_w)$
- **Loss Functions:**
  - MSE Loss for position
  - Quaternion loss for orientation normalization
- **Hyperparameters:**
  - Batch Size: 8
  - Optimizer: Adam
  - Learning Rate:  $1 \times 10^{-4}$
  - Epochs: 50
  - Dropout: 0.3
- **Normalization:** Quaternion normalization during training to ensure valid orientation predictions

## 7.5 Active Frame Sampling Strategy

- Fixed stride (every 10<sup>th</sup> frame) to simulate exploration
- Alternate approach considered: frame selection via pixel-wise difference (visual novelty)
- Downsampling for computational efficiency and to simulate limited onboard resources

## 7.6 Evaluation Techniques

- Trajectory smoothing using Gaussian filters
- Kabsch algorithm for rigid alignment
- Absolute Trajectory Error (ATE) computation post-alignment
- Visualization of aligned and raw trajectories with marked start/end points

## 8 Evaluation and Results

The model's performance was evaluated using Absolute Trajectory Error (ATE). After applying smoothing and rigid alignment:

- **ATE: 0.0156 meters** (indicating high prediction accuracy).
- The final trajectory plot showed strong relation between predicted and ground truth trajectories post-alignment.
- A 3D visualization highlighted the paths of trajectories, aiding in assessing trajectory consistency.

To better understand localized prediction behavior, ATE was also computed on a per-frame basis. The resulting plot (Figure 3) indicates that the model maintained consistently low error during the early trajectory (frames 0–300), with an error dip around frame 390 representing exceptionally accurate predictions. A gradual increase in error is observed after frame 400, likely due to accumulated drift or complex visual scenes. Despite this, the increase remains smooth, indicating stable but slightly diverging predictions.



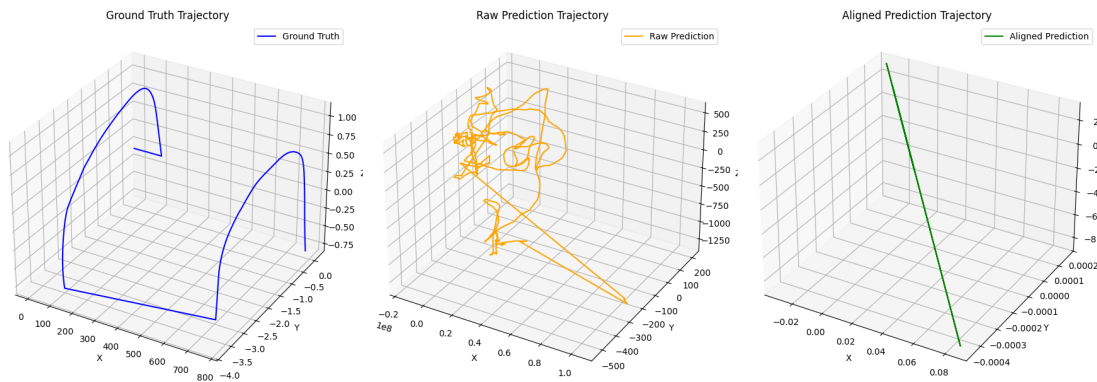


Figure 2: Aligned predicted, raw predicted and ground truth trajectories after alignment.

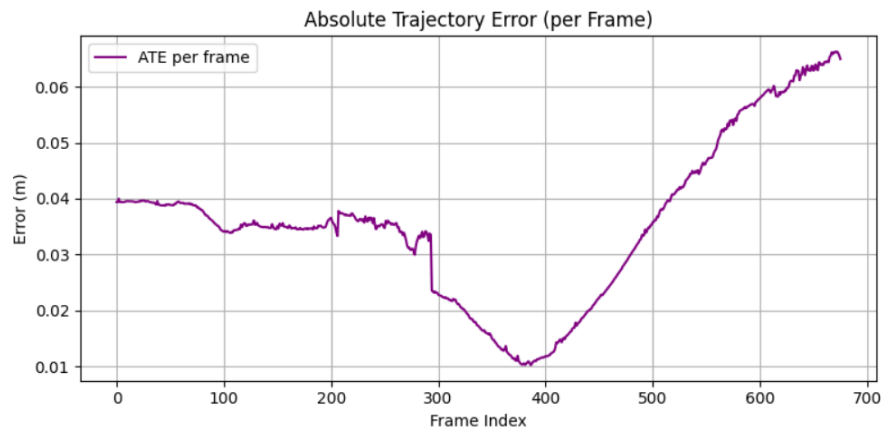


Figure 3: Absolute Trajectory Error (ATE) per frame across the trajectory.

Challenges encountered included:

- Noisy intermediate predictions.
- Poor initial alignment between predicted and ground truth poses.

These issues were effectively mitigated using:

- Gaussian smoothing for trajectory refinement.
- Rigid alignment via the Kabsch algorithm to correct rotational and translational discrepancies.

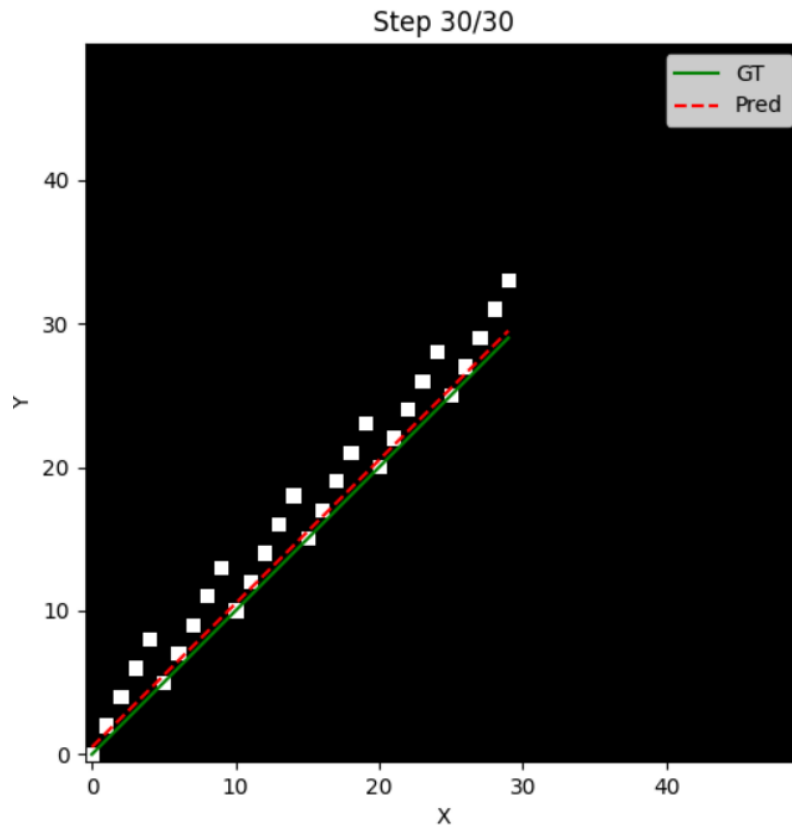


Figure 4: 2D visualization of predicted vs. ground truth trajectories with start/end points marked by using active SLAM.

## 9 Conclusion

This project successfully demonstrates a practical implementation of Neural SLAM using deep learning techniques to estimate agent poses from RGB-D data. The achieved ATE of 0.0156 meters signifies high localization accuracy. The integration of trajectory post-processing and simplified Active SLAM strategies further enhanced the effectiveness and interpretability of the system. This work lays a foundation for future extensions such as incorporating dynamic object detection, using reinforcement learning for real-time action decisions in Active SLAM, and expanding to full 3D semantic mapping for higher-level robotic understanding.

## References

- [1] L. Huang et al. “Invited: Algorithm and Hardware Co-Design for Energy-Efficient Neural SLAM”. In: *ACM* (2024). DOI: 10.1145/3649329.3664829.
- [2] A. G. Gelen and A. Atasoy. “An Artificial Neural SLAM Framework for Event-Based Vision”. In: *IEEE Access* 11 (2023). DOI: 10.1109/access.2023.3282637.
- [3] M. N. Favorskaya. “Deep Learning for Visual SLAM: The State-of-the-Art and Future Trends”. In: *Electronics* 12.9 (2023). DOI: 10.3390/electronics12092006.
- [4] K. Yang et al. “SLAM Meets NeRF: A Survey of Implicit SLAM Methods”. In: *World Electric Vehicle Journal* 15 (2024). DOI: 10.3390/wevj15030085.
- [5] Abhinav Gupta. “Learning to Explore using Active Neural SLAM”. In: *Conference Paper*. 2020.
- [6] F. F. R. Merveille et al. “Enhancing Underwater SLAM Navigation and Perception: A Comprehensive Review of Deep Learning Integration”. In: *Sensors* 24 (2024). DOI: 10.3390/s24217034.